Linux Desktop Chat
Manual

March 20, 2026

Contents

1 Einleitung **6**
1.1 docs_manual . . . 6
1.1.1 Wurzel . . . 6
1.1.2 modules/ . . . 6
1.1.3 roles/ . . . 6
1.1.4 workflows/ . . . 7
1.1.5 templates/ . . . 7
1.2 Architecture . . . 7
1.3 Standards . . . 7

2 Module **8**
2.1 Agents . . . 8
2.1.1 1. Fachsicht . . . 8
2.1.2 2. Rollenmatrix . . . 8
2.1.3 3. Prozesssicht . . . 8
2.1.3.1 3.1 Chat mit Agentenprofil . . . 8
2.1.3.2 3.2 Delegation (Einstieg Slash-Command) . . . 9
2.1.4 4. Interaktionssicht . . . 9
2.1.5 5. Fehler- / Eskalationssicht . . . 9
2.1.6 6. Wissenssicht . . . 10
2.1.7 7. Perspektivwechsel . . . 10
2.2 Chains . . . 10
2.2.1 1. Fachsicht . . . 10
2.2.2 2. Rollenmatrix . . . 10
2.2.3 3. Prozesssicht . . . 11
2.2.3.1 3.1 Kontext-Override-Kette . . . 11
2.2.3.2 3.2 policy_chain (Explainability) . . . 11
2.2.3.3 3.3 Delegation . . . 11
2.2.3.4 3.4 Pipeline-Engine . . . 11
2.2.4 4. Interaktionssicht . . . 11
2.2.5 5. Fehler- / Eskalationssicht . . . 12
2.2.6 6. Wissenssicht . . . 12
2.2.7 7. Perspektivwechsel . . . 12

2.3 Chat . . . 12
2.3.1 1. Fachsicht . . . 12
2.3.2 2. Rollenmatrix . . . 12
2.3.3 3. Prozesssicht . . . 13
2.3.4 4. Interaktionssicht . . . 13
2.3.5 5. Fehler- / Eskalationssicht . . . 14
2.3.6 6. Wissenssicht . . . 14
2.3.7 7. Perspektivwechsel . . . 14
2.4 Context . . . 15
2.4.1 1. Fachsicht . . . 15
2.4.2 2. Rollenmatrix . . . 15
2.4.3 3. Prozesssicht . . . 15
2.4.3.1 3.1 Context Mode (`ChatContextMode`) . . . 15
2.4.3.2 3.2 Detail Level (`ChatContextDetailLevel`) 16
2.4.3.3 3.3 Include-Felder (Persistenz-Keys) . . . 16
2.4.3.4 3.4 Profile (`ChatContextProfile`) . . . 16
2.4.3.5 3.5 Overrides (Priorit\{}"atskette) . . . 16
2.4.3.6 3.6 Rendering und Limits . . . 17
2.4.4 4. Interaktionssicht . . . 17
2.4.5 5. Fehler- / Eskalationssicht . . . 17
2.4.6 6. Wissenssicht . . . 18
2.4.7 7. Perspektivwechsel . . . 18
2.5 GUI . . . 18
2.5.1 1. Fachsicht . . . 18
2.5.2 2. Rollenmatrix . . . 18
2.5.3 3. Prozesssicht . . . 18
2.5.4 4. Interaktionssicht . . . 19
2.5.4.1 4.1 Navigation . . . 19
2.5.4.2 4.2 Workspaces (eingebettete Hauptfl\{}"ache je Screen) . . . 19
2.5.4.3 4.3 Panels . . . 20
2.5.4.4 4.4 Inspector & Backends . . . 21
2.5.5 5. Fehler- / Eskalationssicht . . . 21
2.5.6 6. Wissenssicht . . . 21
2.5.7 7. Perspektivwechsel . . . 21
2.6 Prompts . . . 21
2.6.1 1. Fachsicht . . . 21
2.6.2 2. Rollenmatrix . . . 22
2.6.3 3. Prozesssicht . . . 22
2.6.4 4. Interaktionssicht . . . 22
2.6.5 5. Fehler- / Eskalationssicht . . . 23
2.6.6 6. Wissenssicht . . . 23
2.6.7 7. Perspektivwechsel . . . 23
2.7 Providers . . . 23

2.7.1 1. Fachsicht . . . 23
2.7.2 2. Rollenmatrix . . . 23
2.7.3 3. Prozesssicht . . . 24
2.7.4 4. Interaktionssicht . . . 24
2.7.5 5. Fehler- / Eskalationssicht . . . 24
2.7.6 6. Wissenssicht . . . 25
2.7.7 7. Perspektivwechsel . . . 25
2.8 RAG . . . 25
2.8.1 1. Fachsicht . . . 25
2.8.2 2. Rollenmatrix . . . 25
2.8.3 3. Prozesssicht . . . 25
2.8.4 4. Interaktionssicht . . . 26
2.8.5 5. Fehler- / Eskalationssicht . . . 26
2.8.6 6. Wissenssicht . . . 27
2.8.7 7. Perspektivwechsel . . . 27
2.9 Settings . . . 27
2.9.1 1. Fachsicht . . . 27
2.9.2 2. Rollenmatrix . . . 27
2.9.3 3. Prozesssicht . . . 27
2.9.4 4. Interaktionssicht . . . 28
2.9.4.1 4.1 Acht Kategorien (IDs und UI-Titel) . . . 28
2.9.4.2 4.2 Erweiterung (API) . . . 28
2.9.4.3 4.3 Relevante Keys (Auszug aus `AppSettings`) 28
2.9.5 5. Fehler- / Eskalationssicht . . . 29
2.9.6 6. Wissenssicht . . . 29
2.9.7 7. Perspektivwechsel . . . 29

3 Workflows **30**
3.1 agent_usage . . . 30
3.1.1 Ziel . . . 30
3.1.2 Voraussetzungen . . . 30
3.1.3 Schritte A: Agent im Chat verwenden . . . 30
3.1.4 Schritte B: Agent Tasks \{}"offnen . . . 31
3.1.5 Schritte C: Delegation mit `/delegate` . . . 31
3.1.6 Varianten . . . 31
3.1.7 Fehlerf\{}"alle . . . 31
3.1.8 Tipps . . . 32
3.2 chat_usage . . . 32
3.2.1 Ziel . . . 32
3.2.2 Voraussetzungen . . . 32
3.2.3 Schritte . . . 32
3.2.3.1 A) Chat-Bereich \{}"offnen . . . 32
3.2.3.2 B) Chat ausw\{}"ahlen oder anlegen . . . 33
3.2.3.3 C) Modell pr\{}"ufen . . . 33

3.2.3.4 D) Nachricht schreiben und senden 33
3.2.3.5 E) Rolle nur f\{}"ur die n\{}"achste Frage setzen (Slash-Befehl) 33
3.2.4 Varianten 34
3.2.5 Fehlerf\{}"alle 34
3.2.6 Tipps 34
3.3 context_control 34
3.3.1 Ziel 35
3.3.2 Voraussetzungen 35
3.3.3 Begriffe in drei Stufen (f\{}"ur Endnutzer) 35
3.3.4 Schritte: Kontextmodus \{}"andern 35
3.3.5 Wann **off**? 36
3.3.6 Wann **semantic**? 36
3.3.7 Wann **neutral**? 36
3.3.8 Kontextleiste im Chat (ohne Einstellungen zu \{}"andern) 36
3.3.9 Varianten 37
3.3.10 Fehlerf\{}"alle 37
3.3.11 Tipps 37
3.4 settings_usage 37
3.4.1 Ziel 37
3.4.2 Voraussetzungen 38
3.4.3 Schritte: Einstellungen \{}"offnen 38
3.4.4 Schritte: Kategorien der Reihe nach (8 St\{}"uck) . . 38
3.4.5 Schritte: \{}"Anderung speichern 38
3.4.6 Varianten 39
3.4.7 Fehlerf\{}"alle 39
3.4.8 Tipps 39

4 Rollen **40**
4.1 Admin 40
4.1.1 Aufgaben 40
4.1.2 Typische Workflows 40
4.1.2.1 Erreichbarkeit Ollama pr\{}"ufen 40
4.1.2.2 Standardmodell und Modellliste 41
4.1.2.3 Einstellungskategorien durchgehen (8 St\{}"uck) 41
4.1.2.4 RAG-Space und Top-K 41
4.1.2.5 Kontrolle bei Supportf\{}"allen 42
4.1.3 Genutzte Module 42
4.1.4 Risiken 42
4.1.5 Best Practices 42
4.2 Business 43
4.2.1 Aufgaben 43
4.2.2 Typische Workflows (organisatorisch) 43

4.2.2.1 Einf\{}"uhrung f\{}"ur Wissensarbeit mit internen Dokumenten . . . 43
4.2.2.2 Einf\{}"uhrung f\{}"ur spezialisierte Bearbeitung (Agenten / Delegation) . . . 43
4.2.2.3 Standardisierung von Texten (Prompts) . . . 44
4.2.3 Genutzte Module (Nutzen in Gesch\{}"aftssprache) . 44
4.2.4 Risiken . . . 44
4.2.5 Best Practices . . . 44
4.3 Entwickler . . . 45
4.3.1 Aufgaben . . . 45
4.3.2 Typische Workflows . . . 45
4.3.2.1 Repository lokal starten . . . 45
4.3.2.2 Neuen Screen oder Workspace anbinden . . . 45
4.3.2.3 Settings-Kategorie erg\{}"anzen . . . 46
4.3.2.4 Slash-Command hinzuf\{}"ugen . . . 46
4.3.2.5 Kontext-Debugging . . . 46
4.3.2.6 Provider-/Modellprobleme eingrenzen . . . 46
4.3.2.7 GUI-Pfade nicht mit Alt-Repo verwechseln . 46
4.3.3 Genutzte Module . . . 47
4.3.4 Risiken . . . 47
4.3.5 Best Practices . . . 47
4.4 Fachanwender . . . 48
4.4.1 Aufgaben . . . 48
4.4.2 Typische Workflows . . . 48
4.4.2.1 Neuen Chat starten und schreiben . . . 48
4.4.2.2 Rolle oder Routing per Eingabezeile . . . 48
4.4.2.3 RAG nutzen . . . 49
4.4.2.4 Kontext verstehen (ohne Technik) . . . 49
4.4.2.5 Typische Fehler und was Sie tun k\{}"onnen 50
4.4.3 Genutzte Module . . . 50
4.4.4 Risiken . . . 50
4.4.5 Best Practices . . . 50

Chapter 1

Einleitung

1.1 docs_manual

1.1.1 Wurzel

Pfad	Rolle
architecture.md (`architecture.md`)	Architektur-Anker
standards.md (`standards.md`)	Standards-Anker
modules/ (`modules/`)	Modulbezogene Kapitel
roles/ (`roles/`)	Rollenbezogene Kapitel
workflows/ (`workflows/`)	Ablaufkapitel
templates/ (`templates/`)	Vorlagen

1.1.2 modules/

In jedem Unterordner beschreibt **'README.md'** das Modul (Fach-, Prozess-, Interaktions-, Fehler-, Begriffs-, Perspektivsicht) anhand der Codebasis.

Verzeichnis
chat/ (`modules/chat/`)
context/ (`modules/context/`)
settings/ (`modules/settings/`)
providers/ (`modules/providers/`)
agents/ (`modules/agents/`)
rag/ (`modules/rag/`)
chains/ (`modules/chains/`)
prompts/ (`modules/prompts/`)
gui/ (`modules/gui/`)

1.1.3 roles/

Pro Rolle: **'README.md'** = Rollen-Handbuch (Aufgaben, Workflows, Module, Risiken, Best Practices).

Rolle
fachanwender (`roles/fachanwender/README.md`)
admin (`roles/admin/README.md`)
entwickler (`roles/entwickler/README.md`)
business (`roles/business/README.md`)

1.1.4 workflows/

Endnutzer-Workflows (Schritt-f\{}"ur-Schritt): Ziel, Voraussetzungen, Schritte, Varianten, Fehlerf\{}"alle, Tipps.

Workflow
chat_usage.md (`workflows/chat_usage.md`)
context_control.md (`workflows/context_control.md`)
agent_usage.md (`workflows/agent_usage.md`)
settings_usage.md (`workflows/settings_usage.md`)

1.1.5 templates/

Datei
module_template.md (`templates/module_template.md`)
role_template.md (`templates/role_template.md`)
workflow_template.md (`templates/workflow_template.md`)

1.2 Architecture

1.3 Standards

Chapter 2

Module

2.1 Agents

2.1.1 1. Fachsicht

Das **Agenten-Subsystem** verwaltet **Profile**, **Aufgaben** und **Orchestrierung** (Planner, Delegation, Execution). Nutzer w\{}"ahlen Agenten im Chat oder nutzen **Operations $\{}rightarrow$ Agent Tasks** bzw. **Control Center $\{}rightarrow$ Agents**. Slash-Command **'/delegate'** aktiviert in der Command-Schicht `use_delegation=True` (`app/core/commands/chat_commands.py`); die weitere Ausf\{}"uhrung liegt in den Agenten- und Chat-Pfaden.

Paket: `app/agents/`

2.1.2 2. Rollenmatrix

Rolle	Nutzung
Fachanwender	Agent ausw\{}"ahlen; Agent Tasks; `/delegate <Text>`.
Admin	Persistenz der Agent-Daten (SQLite/Repository-Kontext der App).
Entwickler	`agent_profile.py`, `agent_repository.py`, `planner.py`, `delegation_engine.p`
Business	?Spezialisierte Assistenten und Aufgabenautomatisierung.

2.1.3 3. Prozesssicht

2.1.3.1 3.1 Chat mit Agentenprofil

```
User w\"ahlt Agent (GUI)
        ?
        v
AgentProfile (system_prompt, assigned_model / Rolle)
        ?
        v
Gleicher Chat-Pfad wie Modul Chat $\rightarrow$ Provider
```

2.1.3.2 3.2 Delegation (Einstieg Slash-Command)

```
Eingabe: /delegate <Aufgabe>
        ?
        v
parse_slash_command $\rightarrow$ use_delegation=True, remaining_text=Aufgabe
        ?
        v
ChatService / Agenten-Pipeline
  (Planner $\rightarrow$ Task Graph $\rightarrow$ Delegation Engine $\rightarrow$ Ex
  --- konkrete Aufrufkette in app/agents/*.py
```

Relevante Dateien (Auswahl): `planner.py`, `delegation_engine.py`, `execution_engine.py`, `orchestration_service.py`, `agent_task_runner.py`.

2.1.4 4. Interaktionssicht

UI

- `operations_agent_tasks` $\{}rightarrow$ `AgentTasksWorkspace` (`operations_screen.py`)
- `cc_agents` $\{}rightarrow$ `AgentsWorkspace` (`control_center_screen.py`)
- Chat: Agentenwahl je nach `chat_workspace.py`-Implementierung

Services

- `app/services/agent_service.py`
- Infrastruktur-Registrierung: `agents`, `agent_service` (siehe `docs/SYSTEM_MAP.md`)

Seed / Hilfe

- `app/agents/seed_agents.py` — u. a. f\{}"ur `app/help/doc_generator.py` (`generated_agents`)

2.1.5 5. Fehler- / Eskalationssicht

Problem	Ursache
Leeres Agent-Dropdown	Repository/DB nicht bef\{}"ullt; Seed fehlgeschlagen.
`/delegate` ohne Wirkung	Kein Text nach `/delegate` $\{}rightarrow$ nur Hinweis-`SlashComman`
Task h\{}"angt	Execution/Planner-Fehler; Logs unter Runtime Debug / Agent Activit

2.1.6 6. Wissenssicht

Begriff	Ort
`AgentProfile`	`app/agents/agent_profile.py`
`AgentRepository`	`app/agents/agent_repository.py`
`operations_agent_tasks`, `cc_agents`	Workspace-IDs in `operations_screen.py`, `control_cent`
`use_delegation`	`SlashCommandResult` in `app/core/commands/chat_comma`

2.1.7 7. Perspektivwechsel

Perspektive	Fokus
User	Agent Tasks, Chat-Agent, `/delegate`.
Admin	Datenhaltung Agent-Tabellen.
Dev	Orchestrierungsgraph, Fehlerpfade, Live-Tests `tests/live/`.

2.2 Chains

2.2.1 1. Fachsicht

Im Repository gibt es **kein** Top-Level-Paket oder Produktmodul mit dem Namen **chains**. ?Ketten bezeichnet hier **mehrere unabh\{}"angige Ablaufketten**, die im Code vorkommen:

1. **Kontext-Override-Kette** — feste Priorit\{}"at der Kontext-Konfiguration in `ChatService._resolve_context_configuration` (`app/services/chat_service.py`).
2. **Explainability 'policy_chain'** — serialisierte Darstellung der Kontextentscheidung (`app/context/explainability/context_explanation_serializer.py`, Funktion `_policy_chain_to_dict`).
3. **Delegations-Kette** — Einstieg `/delegate` $\{}rightarrow$ `use_delegation` in `app/core/commands/chat_commands.py` $\{}rightarrow$ Agenten-Pipeline in `app/agents/` (Planner, Delegation, Execution).
4. **Pipeline-Engine** — generische Schritt-Ausf\{}"uhrung in `app/pipelines/` (`engine`, `executors`, `services/pipeline_service.py`); Service-Name `pipeline_service` in `docs/SYSTEM_MAP.md`.

2.2.2 2. Rollenmatrix

Rolle	Nutzung
Fachanwender	`/delegate`; ggf. Pipelines \{}"uber zugeh\{}"orige UI (falls angebunden).
Admin	Keine zentrale ?Chain-Konsole.
Entwickler	Priorit\{}"atsliste in `ChatService`, Serializer, Agenten-Module, `app/pipelines`
Business	Orchestrierung komplexer Aufgaben (Delegation); technische Pipelines getrennt

2.2.3 3. Prozesssicht

2.2.3.1 3.1 Kontext-Override-Kette

Siehe Modul **context** — `_PRIORITY_SOURCES`:

`profile_enabled` $\{}rightarrow$ `explicit_context_policy` $\{}rightarrow$ `chat_default_context_policy` $\{}rightarrow$ `project_default_context_policy` $\{}rightarrow$ `request_context_hint` $\{}rightarrow$ `individual_settings`.

2.2.3.2 3.2 policy_chain (Explainability)

```
ChatService / ContextExplainService
        ?
        v
ContextExplanation / Trace
        ?
        v
context_explanation_serializer._policy_chain_to_dict
        ?
        v
JSON-Felder (deterministische Key-Reihenfolge laut Modul-Docstring)
```

2.2.3.3 3.3 Delegation

```
/parse $\rightarrow$ use_delegation + Text
        ?
        v
Agenten: planner, delegation_engine, execution_engine (app/agents/)
```

2.2.3.4 3.4 Pipeline-Engine

```
PipelineDefinition / Run
        ?
        v
Engine + Step-Executors (Shell, Python, \ldots{} in app/pipelines/executors/)
```

2.2.4 4. Interaktionssicht

Kette	UI
Kontext-Override	Settings $\{}rightarrow$ Advanced (nur Mode); Inspector/Debug
policy_chain	QA-Observability / Introspection (Runtime-Workspaces)
Delegation	Chat-Eingabe `/delegate`
Pipelines	Aufrufer von `app/pipelines/services/pipeline_service.py` (nicht im Mod

2.2.5 5. Fehler- / Eskalationssicht

Problem	Kontext
Manuelle Kontext-Settings unwirksam	H\{}"ohere Priorit\{}"at in Override-Kette aktiv.
policy_chain-Drift in Tests	Serializer-\{}"Anderung; QA-Dokus unter `docs/qa/`.
`/delegate` ohne Aktion	Leerer Resttext nach Command.
Pipeline-Schritt fehlgeschlagen	Executor oder Definition in `app/pipelines/`.

2.2.6 6. Wissenssicht

Begriff	Ort
`_PRIORITY_SOURCES`	`app/services/chat_service.py`
`policy_chain`	`app/context/explainability/context_explanation_ser`
`use_delegation`	`app/core/commands/chat_commands.py`
`PipelineRun`, `PipelineDefinition`	`app/pipelines/models/` (Export in `app/pipelines/__ini`

2.2.7 7. Perspektivwechsel

Perspektive	Fokus
User	`/delegate` verstehen; Kontextmodus.
Admin	—
Dev	Keine Vermischung der vier Ketten; jeweils eigenes Paket lesen.

2.3 Chat

2.3.1 1. Fachsicht

Das **Chat-Modul** umfasst die gef\{}"uhrte Konversation mit einem Sprachmodell \{}"uber Ollama: Sitzungen (Chats), Nachrichtenverlauf, Eingabe, optionales Streaming und Slash-Commands zur Rollen- und Routing-Steuerung. Es verbindet die Oberfl\{}"ache mit `ChatService`, Kontextinjektion (siehe Modul **context**), Modell-/Provider-Pfad und optional RAG-Anreicherung \{}"uber den Knowledge-Pfad.

Nutzen: zentraler Arbeitsbereich f\{}"ur alle textbasierten Modellanfragen innerhalb eines Projekts oder global.

2.3.2 2. Rollenmatrix

Rolle	Nutzung
Fachanwender	Chat-Workspace unter Operations; Sessions, senden, Slash-Commands.
Admin	Erreichbarkeit Ollama/Modelle, ggf. Cloud-Keys (\{}"uber Settings/Control Ce
Entwickler	`app/services/chat_service.py`, `app/gui/domains/operations/chat/`, app/
Business	Nutzung als Produktfunktion ?Chat mit KI; keine zus\{}"atzliche technische Ob

2.3.3 3. Prozesssicht

```
??????????????????????????????????????????????????????????????????????
? GUI: ChatWorkspace $\rightarrow$ Senden (Text)                        ?
??????????????????????????????????????????????????????????????????????
                              v
??????????????????????????????????????????????????????????????????????
? parse_slash_command (app/core/commands/chat_commands.py)     ?
? $\rightarrow$ Rolle, /auto, /cloud, /delegate (use_delegation)        ?
??????????????????????????????????????????????????????????????????????
                              v
??????????????????????????????????????????????????????????????????????
? ChatService: Message-Liste, Kontextaufl\"osung & Injektion    ?
? (Mode off $\rightarrow$ kein Kontextfragment)                         ?
??????????????????????????????????????????????????????????????????????
                              v
??????????????????????????????????????????????????????????????????????
? Chat Guard (optional): assess_async $\rightarrow$ apply_to_messages   ?
? (app/services/chat_service.py $\rightarrow$ _apply_chat_guard)         ?
??????????????????????????????????????????????????????????????????????
                              v
??????????????????????????????????????????????????????????????????????
? Modell-Routing / Orchestrator $\rightarrow$ Provider (Ollama)         ?
? Streaming gesteuert \"uber AppSettings.chat_streaming_enabled    ?
??????????????????????????????????????????????????????????????????????
                              v
??????????????????????????????????????????????????????????????????????
? Antwort-Stream oder komplette Antwort $\rightarrow$ GUI Conversation   ?
??????????????????????????????????????????????????????????????????????
```

Workspace-IDs (Operations): `operations_chat` in `app/gui/domains/operations/operatio` (Eintrag `ChatWorkspace`).

2.3.4 4. Interaktionssicht

UI

- `app/gui/domains/operations/chat/chat_workspace.py` – Orchestrierung Session-Explorer, Konversation, Eingabe.

- `app/gui/domains/operations/chat/panels/chat_context_bar.py` – Anzeige Projekt / Chat / Topic; Klicks und Kontextmen\{}"u (Navigation, keine Kontext-Modus-Logik).
- `app/gui/chat_backend.py` – Zugriff der GUI auf Backend-Funktionen (Modelle, Senden, Status).

Services / Commands

- `app/services/chat_service.py` – Sendepfad, Kontext, Guard, Streaming.
- `app/core/commands/chat_commands.py` – `ROLE_COMMANDS`, `parse_slash_command`, `SLASH_COMMANDS`.

Relevante Settings-Keys (Auszug)

- `chat_streaming_enabled` – Streaming ein/aus (`AppSettings` in `app/core/config/settings.`

2.3.5 5. Fehler- / Eskalationssicht

Problem	technische Ursache (Code)
Keine Antwort / h\{}"angt	Ollama nicht erreichbar; Provider-Timeout (siehe Provi
Doppeltes Senden / Konflikt bei Stream	Chat Guard und Sendelogik in `ChatService`; parallele S
Slash-Command wirkt nicht	Zeile beginnt nicht mit `/` oder unbekannter Befehl $\{}r
`/delegate` ohne Text	`parse_slash_command`: `consumed=True`, nur Verwendur
Kontext fehlt	`ChatContextMode.OFF` oder h\{}"oherpriore Kontext-O
Unerwartete Prompt-H\{}"artung	Chat Guard bewertet letzte User-Nachricht und modifiz

2.3.6 6. Wissenssicht

Begriff	Quelle
`ChatWorkspace`	`app/gui/domains/operations/chat/chat_workspace.`
`ChatService`	`app/services/chat_service.py`
`SlashCommandResult`, `use_delegation`	`app/core/commands/chat_commands.py`
`chat_streaming_enabled`	`app/core/config/settings.py`
`operations_chat`	Workspace-ID in `operations_screen.py`

2.3.7 7. Perspektivwechsel

Perspektive	Fokus
User	Operations $\{}rightarrow$ Chat; Slash-Commands; Modellwahl in der Chat-Oberfl
Admin	Ollama-Dienst, Modelle, Netz; keine Chat-spezifische Admin-Konsole.
Dev	`ChatService`, Guard-Hooks, Workspace-IDs, Signal/Async-Pfade in `chat_workspac`

2.4 Context

2.4.1 1. Fachsicht

Context beschreibt, wie **Projekt-, Chat- und Topic-Metadaten** als strukturierter Text in den Modell-Prompt einflie\{}ss{}en (oder bewusst ausbleiben). Umfasst im Code vor allem:

- **Darstellung / Fragment:** `app/chat/context.py` (`ChatRequestContext`, `ChatContextRenderOptions`, Limits).
- **Enums und Presets:** `app/core/config/chat_context_enums.py`, `app/chat/context_profiles.py`, `ChatContextProfile` in `app/core/config/settings.py`.
- **Aufl\{}"osung im Chat:** `ChatService._resolve_context_configuration` in `app/services/chat_service.py`.
- **Explainability / Replay / Repro:** `app/context/` (z. B. Serializer, Replay-Services, Registry).

Nutzen: reproduzierbare, einstellbare Kontextinjektion ohne Provider- oder UI-Logik im Renderer (`context.py` importiert kein Settings-Modul f\{}"ur Gesch\{}"aftsregeln; Enums liegen separat).

2.4.2 2. Rollenmatrix

Rolle	Nutzung
Fachanwender	Context Mode in **Settings $\{}rightarrow$ Advanced** (`AdvancedSettingsP`
Admin	Persistenz \{}"uber dasselbe Settings-Backend wie die restliche App.
Entwickler	Aufl\{}"osung, Traces, Replay/Repro-CLI unter `app/cli/`, Tests unter `tests/`.
Business	Steuerung ?wie viel Hintergrund dem Modell mitgegeben wird.

2.4.3 3. Prozesssicht

2.4.3.1 3.1 Context Mode (`ChatContextMode`)

Definiert in `app/core/config/chat_context_enums.py`:

Wert (`str`)	Enum	Wirkung
`off`	`OFF`	Keine Injektion des strukturierten Chat-Kontexts im Service-Pfad.
`neutral`	`NEUTRAL`	Fragment \{}"uber `to_system_prompt_fragment` in neutraler Form.
`semantic`	`SEMANTIC`	Fragment in semantischer Form.

Ung\{}"ultige Werte in `AppSettings` werden bei `get_chat_context_mode()` durch **'SEMANTIC'** ersetzt.

2.4.3.2 3.2 Detail Level (`ChatContextDetailLevel`)

Wert	Enum
`minimal`	`MINIMAL`
`standard`	`STANDARD`
`full`	`FULL`

Ung\{}"ultige Werte $\{}rightarrow$ **'STANDARD'** (`get_chat_context_detail_level()`).

2.4.3.3 3.3 Include-Felder (Persistenz-Keys)

In `AppSettings.load()`:

- `chat_context_include_project` (bool, Default **True**)
- `chat_context_include_chat` (bool, Default **True**)
- `chat_context_include_topic` (bool, Default **False**)

Sie steuern `ChatContextRenderOptions` bei der Aufl\{}"osung.

2.4.3.4 3.4 Profile (`ChatContextProfile`)

Enum in `app/core/config/settings.py`:

- `strict_minimal`, `balanced`, `full_guidance`

Aufl\{}"osung \{}"uber `resolve_chat_context_profile()` in `app/chat/context_profiles.py` $\{}rightarrow$ festeres Paket aus Mode, Detail, Include-Flags.

Schalter: `chat_context_profile_enabled` (bool). Profilname: `chat_context_profile` (String, ung\{}"ultig $\{}rightarrow$ `balanced`).

2.4.3.5 3.5 Overrides (Priorit\{}"atskette)

`ChatService._resolve_context_configuration` dokumentiert die Reihenfolge in `_PRIORITY_SOURCES`:

1. `profile_enabled` — wenn `chat_context_profile_enabled`, Werte aus Profil-Aufl\{}"osung
2. `explicit_context_policy` — Laufzeit-Policy
3. `chat_default_context_policy`
4. `project_default_context_policy`
5. `request_context_hint` — Map in `resolve_profile_for_request_hint()` (`app/chat/context_profiles.py`)

6. `individual_settings` — direkt `get_chat_context_mode()`, `get_chat_context_detail_leve`
   Include-Flags aus `AppSettings`

Die niedrigste Index-Quelle, die Werte liefert, gewinnt nach der Implementationslogik der Methode (Kandidatenliste / `winning_idx`).

2.4.3.6 3.6 Rendering und Limits

- `ChatRequestContext.to_system_prompt_fragment(...)` — ruft bei `NEUTRAL`/`SEMANTIC` intern `_to_neutral_fragment` bzw. `_to_semantic_fragment` auf; **'OFF' darf hier nicht landen** (ValueError laut Docstring).
- `app/chat/context_limits.py` — Zeichen- und Zeilenlimits; leere oder nutzlose Fragmente werden verworfen (`_failsafe_info`, Header-only entfernt).

2.4.4 4. Interaktionssicht

UI

- `app/gui/domains/settings/panels/advanced_settings_panel.py` — gebunden: **'chat_context_mode'** (Combo `neutral` / `semantic` / `off`), `context_debug_enabled`, `debug_panel_enabled`.
- **Keine** weiteren Settings-Panel-Bindings f\{}"ur `chat_context_detail_level`, Include-Flags oder Profil in `app/gui/domains/settings/` (Stand: Volltextsuche nach `chat_context_detail` / `chat_context_include` / `chat_context_profile` in diesem Paket ohne Treffer au\{}ss{}erhalb von `advanced_settings_panel` f\{}"ur `chat_context_mode`).

Services

- `app/services/chat_service.py` — Injektion, Aufl\{}"osung, Traces.
- `app/services/context_explain_service.py` — Erkl\{}"arungen f\{}"ur Observability.

API / CLI

- `app/cli/context_replay.py`, `context_repro_run.py`, `context_repro_batch.py`, Registry-CLI — deterministische Kontext-Replays ohne GUI.

2.4.5 5. Fehler- / Eskalationssicht

Problem	Ursache (Code)
Kein Kontext trotz Erwartung	`ChatContextMode.OFF` oder h\{}"oherpri
Manueller Detail-Level \{}"andert sich nicht in der UI	Kein Formularfeld daf\{}"ur in Settings-P
Fragment leer	`is_empty()`, alle Includes false, oder Fails
Trace/JSON-Drift in QA	Serializer / Explainability; siehe `app/cont`

2.4.6 6. Wissenssicht

Begriff	Definition / Ort
`ChatContextMode`	`app/core/config/chat_context_enums.py`
`ChatContextDetailLevel`	`app/core/config/chat_context_enums.py`
`ChatContextProfile`	`app/core/config/settings.py`
`ResolvedChatContextConfig`	`app/chat/context_profiles.py`
`ChatContextResolutionTrace`	`app/chat/context_profiles.py` (`source`, `profile`, `mode`, `deta`
`ChatRequestContext`	`app/chat/context.py`
`ChatContextRenderOptions`	`app/chat/context.py`
`policy_chain` (Explainability)	u. a. `app/context/explainability/context_explanation_ser`

2.4.7 7. Perspektivwechsel

Perspektive	Fokus
User	Advanced: Kontextmodus; ChatContextBar; erwartet ggf. wenig Kontext bei `off`.
Admin	Keine separate Kontext-Konsole; Backup/Wiederherstellung der Settings-Datenbank
Dev	`_resolve_context_configuration`, Replay-JSON, Governance-Dokus unter `docs/C`

2.5 GUI

2.5.1 1. Fachsicht

Die **GUI** ist die **PySide6-Shell** der Anwendung: Hauptfenster, Bereichsumschaltung, Workspaces je Screen, **Navigation** (Sidebar, Breadcrumbs, Command Palette), **Inspector** (rechts), **Monitors** (unten) und dom\{}"anenspezifische **Panels**. Zentraler Code unter **‘app/gui/‘** (nicht `app/ui/` — dieses Verzeichnis existiert im aktuellen Tree nicht, vgl. `docs/DOC_GAP_ANALYSIS.md`).

2.5.2 2. Rollenmatrix

Rolle	Nutzung
Fachanwender	Bedienung aller Screens und Workspaces.
Admin	Kein separates Admin-UI; Konfiguration \{}"uber Settings.
Entwickler	`app/gui/bootstrap.py`, Domains, WorkspaceHost, Inspector, Tests `tests/ui/`
Business	Produktoberfl\{}"ache ?Linux Desktop Chat.

2.5.3 3. Prozesssicht

```
ShellMainWindow (app/gui/shell/)
        ?
        ??? NavigationSidebar $\rightarrow$ NavArea wechseln
        ?
```

```
??? WorkspaceHost $\rightarrow$ Screen aus ScreenRegistry
?
??? InspectorHost (rechts)
?
??? Monitors (unten, app/gui/monitors/)
```

Screen-Registrierung beim Start: `register_all_screens()` in `app/gui/bootstrap.py` — mappt jede `NavArea`-ID auf genau eine Screen-Klasse:

`NavArea` (Konstante)	Screen-Klasse	Anzeigename (Registry-Titel)
`command_center`	`DashboardScreen`	Kommandozentrale
`project_hub`	`ProjectHubScreen`	Project Hub
`operations`	`OperationsScreen`	Operations
`control_center`	`ControlCenterScreen`	Control Center
`qa_governance`	`QAGovernanceScreen`	QA & Governance
`runtime_debug`	`RuntimeDebugScreen`	Runtime / Debug
`settings`	`SettingsScreen`	Settings

Definition der Konstanten: `app/core/navigation/nav_areas.py`.

2.5.4 4. Interaktionssicht

2.5.4.1 4.1 Navigation

- **Sidebar / Bereiche:** `app/gui/navigation/` (`NavigationSidebar`, `nav_areas`).
- **Command Palette:** `app/gui/navigation/command_palette.py` — Befehle aus `app/gui/commands/` (`CommandRegistry`).
- **Breadcrumbs:** `app/gui/breadcrumbs.py` (von Screens bei Workspace-Wechsel angesprochen).

2.5.4.2 4.2 Workspaces (eingebettete Hauptfl\{}"ache je Screen)

Operations (`operations_screen.py`) — IDs:

Workspace-ID	Klasse
`operations_projects`	`ProjectsWorkspace`
`operations_chat`	`ChatWorkspace`
`operations_agent_tasks`	`AgentTasksWorkspace`
`operations_knowledge`	`KnowledgeWorkspace`
`operations_prompt_studio`	`PromptStudioWorkspace`

Standard nach Start des Stacks: `operations_projects` (`set_current` in `_setup_ui`).

Control Center (`control_center_screen.py`):

Workspace-ID	Klasse
`cc_models`	`ModelsWorkspace`
`cc_providers`	`ProvidersWorkspace`
`cc_agents`	`AgentsWorkspace`
`cc_tools`	`ToolsWorkspace`
`cc_data_stores`	`DataStoresWorkspace`

Standard: `cc_models`.
QA & Governance (`qa_governance_screen.py`):

Workspace-ID	Klasse
`qa_test_inventory`	`TestInventoryWorkspace`
`qa_coverage_map`	`CoverageMapWorkspace`
`qa_gap_analysis`	`GapAnalysisWorkspace`
`qa_incidents`	`IncidentsWorkspace`
`qa_replay_lab`	`ReplayLabWorkspace`

Standard: `qa_test_inventory`.
Runtime / Debug (`runtime_debug_screen.py`):

Workspace-ID	Klasse
`rd_introspection`	`IntrospectionWorkspace`
`rd_qa_cockpit`	`QACockpitWorkspace`
`rd_qa_observability`	`QAObservabilityWorkspace`
`rd_eventbus`	`EventBusWorkspace`
`rd_logs`	`LogsWorkspace`
`rd_metrics`	`MetricsWorkspace`
`rd_llm_calls`	`LLMCallsWorkspace`
`rd_agent_activity`	`AgentActivityWorkspace`
`rd_system_graph`	`SystemGraphWorkspace`

Standard: `rd_introspection`.

Settings — acht Kategorien \{}"uber `SettingsWorkspace` (siehe Modul **settings**), nicht als `QStackedWidget`-Liste in einem separaten `*Screen` wie oben, sondern \{}"uber `settings_workspace.py` + `SettingsNavigation`.

2.5.4.3 4.3 Panels

Panels sind komponentenweise UI-Teile innerhalb von Workspaces (z. B. unter `app/gui/domains/operations/chat/panels/` — `ChatConversationPanel`, `ChatInputPanel`, `ChatContextBar`, `chat_navigation_panel`, \{}ldots{}).
Es gibt keine einzelne zentrale ?Panel-Registry-Datei; die Einbindung erfolgt \{}"uber die jeweilige `*Workspace`-Klasse.

2.5.4.4 4.4 Inspector & Backends

- **Inspector:** `app/gui/inspector/` — z. B. `ChatContextInspector` f\{}"ur Chat-Kontext.
- **ChatBackend / KnowledgeBackend:** `app/gui/chat_backend.py`, `app/gui/knowledge_backend.py` — Anbindung an Services.

2.5.5 5. Fehler- / Eskalationssicht

Problem	Ursache
Screen leer nach Navigation	Factory fehlt in `register_all_screens` oder Exception in Screen-_
Inspector zeigt alten Inhalt	`prepare_for_setup` / `content_token` in `OperationsScreen`/`Runti`
Falscher Workspace aktiv	`show_workspace` nicht aufgerufen; Breadcrumb-Drift.

2.5.6 6. Wissenssicht

Begriff	Ort
`WorkspaceHost`	`app/gui/workspace/workspace_host.py`
`ScreenRegistry`	`app/gui/workspace/screen_registry.py`
`register_all_screens`	`app/gui/bootstrap.py`
`NavArea`	`app/core/navigation/nav_areas.py`
`BaseScreen`	`app/gui/shared/base_screen.py`

2.5.7 7. Perspektivwechsel

Perspektive	Fokus
User	Sidebar-Struktur wie in `docs/00_map_of_the_system.md` ?4.
Admin	—
Dev	Keine Suche unter `app/ui/`; GUI-Code nur unter `app/gui/`.

2.6 Prompts

2.6.1 1. Fachsicht

Prompts sind **wiederverwendbare Textbausteine** (Vorlagen), verwaltet \{}"uber das Paket **'app/prompts/'** mit Repository und Speicher-Backends. Die Hauptoberfl\{}"ache ist **Operations $\{}rightarrow$ Prompt Studio** (`PromptStudioWorkspace`). Speicherort und L\{}"oschverhalten steuern **'AppSettings'**: `prompt_storage_type` (`database` oder Verzeichnis), `prompt_directory`, `prompt_confirm_delete`.

2.6.2 2. Rollenmatrix

Rolle	Nutzung
Fachanwender	Prompt Studio: anlegen, bearbeiten, auf Chat anwenden.
Admin	Schreibrechte auf `prompt_directory`; SQLite-Datei `chat_history.db` bei DB-M
Entwickler	`prompt_service.py`, `prompt_repository.py`, `storage_backend.py`, `prompt_m`
Business	Standardantworten und Arbeitsvorlagen.

2.6.3 3. Prozesssicht

```
GUI: PromptStudioWorkspace
        ?
        v
PromptService (app/prompts/prompt_service.py)
        ?
        v
PromptRepository + StorageBackend
  (database vs. directory laut AppSettings)
        ?
        v
Persistenz (SQLite oder Dateisystem)
```

Service-Registrierung: Name `prompts` in der Infrastruktur (`docs/SYSTEM_MAP.md`).

2.6.4 4. Interaktionssicht

UI

- Workspace-ID: **‘operations_prompt_studio‘** (`operations_screen.py`)
- Datei: `app/gui/domains/operations/prompt_studio/prompt_studio_workspace.py`
- Inspector: `app/gui/inspector/` (kontextbezogen f\{}"ur Prompt Studio, siehe `docs/00_map_of_the_system.md`)

Settings

- `prompt_storage_type`, `prompt_directory`, `prompt_confirm_delete` — `app/core/config/settings.py`

Integration Chat

- Beide Workspaces liegen unter `app/gui/domains/operations/` (`ChatWorkspace`, `PromptStudioWorkspace`); die konkrete Kopplung (Signale/Methoden) steht in diesen Modulen und zugeh\{}"origen Panels.

2.6.5 5. Fehler- / Eskalationssicht

Problem	Ursache
Speichern fehlgeschlagen	Backend-Fehler, Pfad nicht beschreibbar, DB gesperrt.
Prompts ?weg nach Umstellung	Wechsel `prompt_storage_type`/`prompt_directory` ohne Migrat
Leere Liste	Falsches Verzeichnis oder leere Datenbank.

2.6.6 6. Wissenssicht

Begriff	Ort
`PromptStudioWorkspace`	`app/gui/domains/operations/prompt_studio/`
`operations_prompt_studio`	Workspace-ID
`PromptRepository`	`app/prompts/prompt_repository.py`
`StorageBackend`	`app/prompts/storage_backend.py`

2.6.7 7. Perspektivwechsel

Perspektive	Fokus
User	Prompt Studio, Anwendung im Chat.
Admin	Speicherort, Backups.
Dev	Kontrakttests Prompts unter `tests/contracts/`, Integration `tests/integration/t`

2.7 Providers

2.7.1 1. Fachsicht

Provider kapseln die **HTTP-Anbindung an Ollama** f\{}"ur Chat-Completions: lokal (Standardhost) und optional **Cloud** mit API-Key. Die GUI zeigt Status und Endpunkt im **Control Center $\{}rightarrow$ Providers**; die Auswahl des konkreten Provider-Pfads erfolgt \{}"uber Modell-/Chat-Services und Orchestrator, nicht durch direkte UI$\{}rightarrow$Provider-Kopplung.

Code-Paket: `app/providers/`

2.7.2 2. Rollenmatrix

Rolle	Nutzung
Fachanwender	Sieht Online/Offline und Provider-Infos im Workspace `cc_providers`.
Admin	Ollama-Dienst, Firewall, `ollama_api_key`, Flags `cloud_escalation`, `cloud_via`
Entwickler	`base_provider.py`, `local_ollama_provider.py`, `cloud_ollama_provider.py`,
Business	?Anbindung ans Modell-Backend.

2.7.3 3. Prozesssicht

```
ChatService / Orchestrator
        ?
        v
BaseProvider-Implementierung
  ??? LocalOllamaProvider  (app/providers/local_ollama_provider.py)
  ??? CloudOllamaProvider (app/providers/cloud_ollama_provider.py)
        ?
        v
Ollama HTTP (ollama_client.py)
        ?
        v
Modellantwort (Stream oder komplett) $\rightarrow$ ChatService $\rightarrow$ GUI
```

2.7.4 4. Interaktionssicht

UI

- `app/gui/domains/control_center/workspaces/providers_workspace.py`
- Panels unter `app/gui/domains/control_center/panels/providers_panels.py`
- Workspace-ID: **‘cc_providers‘** (`control_center_screen.py`)

Services

- `app/services/provider_service.py` (in `docs/SYSTEM_MAP.md` als Service aufgef\{}"uhrt)
- Nutzung aus Chat-/Model-Pfad (`app/services/chat_service.py`, Model-Orchestrierung)

Konfiguration (‘AppSettings‘)

- `cloud_escalation`, `cloud_via_local`, `ollama_api_key` u. a.

2.7.5 5. Fehler- / Eskalationssicht

Problem	Ursache
Offline-Status	`ollama serve` nicht aktiv oder falscher Port.
Cloud ohne Antwort	`ollama_api_key` leer/ung\{}"ultig oder `cloud_escalation` aus.
Timeout	Netzwerk, gro\{}ss{}es Modell; siehe Tests unter `tests/chaos/`, `tests/int`

2.7.6 6. Wissenssicht

Begriff	Datei
`BaseProvider`	`app/providers/base_provider.py`
`LocalOllamaProvider`	`app/providers/local_ollama_provider.py`
`CloudOllamaProvider`	`app/providers/cloud_ollama_provider.py`
`Ollama`-Client	`app/providers/ollama_client.py`
`cc_providers`	Workspace-ID in `control_center_screen.py`

2.7.7 7. Perspektivwechsel

Perspektive	Fokus
User	Providers-Workspace: Refresh, Status.
Admin	Endpoint, Keys, Erreichbarkeit.
Dev	Fehlerbehandlung im Client, Aggregation im Orchestrator.

2.8 RAG

2.8.1 1. Fachsicht

RAG (Retrieval-Augmented Generation) liefert **Chunks aus indexierten Dokumenten** als zus\{}"atzlichen Kontext f\{}"ur Modellanfragen. Implementierung im Paket **‘app/rag/‘** mit ChromaDB-Anbindung (`vector_store.py`, `retriever.py`, `context_builder.py`, `rag_pipeline.py`, \{}ldots{}). Die GUI pflegt Quellen und Indexierung unter **Operations $\{}rightarrow$ Knowledge**; Schalter wie `rag_enabled` und `rag_space` liegen in **‘AppSettings‘**.

2.8.2 2. Rollenmatrix

Rolle	Nutzung
Fachanwender	Knowledge-Workspace; RAG in Chat aktivieren (wenn UI/Settings es erlauben)
Admin	`chromadb`-Installation, Verzeichnis `./chroma_db` (typisch), Embedding-Modell in
Entwickler	`app/rag/*.py`, `app/services/knowledge_service.py`, `app/gui/knowledge_ba`
Business	?Wissensbasierte Antworten.

2.8.3 3. Prozesssicht

```
User-Query (Chat)
        ?
        v
KnowledgeService / RAG-Pfad (wenn rag_enabled)
        ?
        v
```

```
Retriever $\rightarrow$ Vector Store (ChromaDB)
        ?
        v
context_builder / rag_pipeline $\rightarrow$ angereicherter Prompt
        ?
        v
LLM (gleicher Pfad wie Chat)
```

Wichtige Module: `service.py`, `retriever.py`, `vector_store.py`, `context_builder.py`, `embedding_service.py`, `document_loader.py`, `chunker.py`, `knowledge_space.py`.

2.8.4 4. Interaktionssicht

UI

- `operations_knowledge` $\{}rightarrow$ `KnowledgeWorkspace` (`operations_screen.py`)
- Backend-Br\{}"ucke: `app/gui/knowledge_backend.py`

Services

- `app/services/knowledge_service.py`
- Registrierter Service-Name `rag` / `knowledge_service` (siehe `docs/SYSTEM_MAP.md`)

Settings (‘AppSettings‘)

- `rag_enabled`, `rag_space`, `rag_top_k`, `self_improving_enabled`, `chat_mode` (RAG-bezogen je nach Verwendung im Code)

Externe Abh\{}"angigkeit

- Paket `chromadb` in `requirements.txt`
- Optional: `scripts/index_rag.py` f\{}"ur Indexierung (referenziert in `help/troubleshooting/troubleshooting.md`)

2.8.5 5. Fehler- / Eskalationssicht

Problem	Ursache
Keine Treffer	Leerer Index, falscher `rag_space`, Embedding-Modell fehlt.
ChromaDB-Fehler	Paket nicht installiert, korruptes Datenverzeichnis.
Hohe Latenz	Gro\{}ss{}es `rag_top_k` oder gro\{}ss{}e Sammlung.

2.8.6 6. Wissenssicht

Begriff	Ort
`KnowledgeWorkspace`	`app/gui/domains/operations/knowledge/knowledge_wo`
`operations_knowledge`	Workspace-ID
`vector_store`, `retriever`	`app/rag/`
`rag_enabled`, `rag_space`, `rag_top_k`	`app/core/config/settings.py`

2.8.7 7. Perspektivwechsel

Perspektive	Fokus
User	Knowledge-Workspace, RAG-Toggle in Chat/Settings.
Admin	Index-Pipeline, Speicherplatz, Ollama-Embeddings.
Dev	Kontrakttests RAG, Failure-Mode-Tests unter `tests/failure_modes/`.

2.9 Settings

2.9.1 1. Fachsicht

Settings b\{}"undelt **persistente Anwendungskonfiguration**: Modellparameter, Thema, RAG, Prompt-Speicher, Chat-Streaming, Kontext-Keys und weitere Flags. Die fachliche Logik liegt in **'AppSettings'** (`app/core/config/settings.py`) mit austauschbarem **'SettingsBackend'** (`app/core/config/settings_backend.py`, u. a. `InMemoryBackend`). Die **GUI** bietet einen Vollbild-Dialog mit **acht** linken Kategorien (`app/gui/domains/settings/navigation.py`, `settings_workspace.py`).

2.9.2 2. Rollenmatrix

Rolle	Nutzung
Fachanwender	Sidebar $\{}rightarrow$ Settings; Kategorien durchklicken; \{}"Anderungen we
Admin	QSettings-Organisation `OllamaChat` / `LinuxDesktopChat` (siehe Hilfe `help/set`
Entwickler	Keys in `AppSettings.load/save`; `register_settings_category_widget`, `regi`
Business	?Konfiguration der Desktop-App ohne technische Schl\{}"usselnamen-Pflicht.

2.9.3 3. Prozesssicht

```
??????????????????????????????????????????
? User \"offnet SettingsScreen              ?
? $\rightarrow$ SettingsWorkspace                     ?
?   Links: SettingsNavigation (Kategorien) ?
?   Mitte: Category-Widget (Stack)        ?
?   Rechts: optional SettingsHelpPanel     ?
??????????????????????????????????????????
```

```
                    v
??????????????????????????????????????????
? Category-Panel liest/schreibt           ?
? get_infrastructure().settings           ?
? (AppSettings) $\rightarrow$ backend.setValue        ?
??????????????????????????????????????????
```

Legacy-Mapping: `SettingsScreen.show_workspace` mappt alte IDs auf neue Kategorie-IDs (`app/gui/domains/settings/settings_screen.py`, `_WORKSPACE_TO_CATEGORY`).

2.9.4 4. Interaktionssicht

2.9.4.1 4.1 Acht Kategorien (IDs und UI-Titel)

Aus `DEFAULT_CATEGORIES` in `app/gui/domains/settings/navigation.py`, Widgets aus `_category_factories` in `settings_workspace.py`:

ID	Titel (UI)	Widget-Klasse
`settings_application`	Application	`ApplicationCategory`
`settings_appearance`	Appearance	`AppearanceCategory`
`settings_ai_models`	AI / Models	`AIModelsCategory`
`settings_data`	Data	`DataCategory`
`settings_privacy`	Privacy	`PrivacyCategory`
`settings_advanced`	Advanced	`AdvancedCategory`
`settings_project`	Project	`ProjectCategory`
`settings_workspace`	Workspace	`WorkspaceCategory`

Hinweis: Die generierte `docs/SYSTEM_MAP.md` kann unter Settings andere Workspace-Namen listen; die **Navigation + Registry im Code** oben sind die referenzierte Wahrheit f\{}"ur die **8** Kategorien.

2.9.4.2 4.2 Erweiterung (API)

- `register_settings_category_widget(category_id, widget_class)` — `settings_workspace.py`
- `register_settings_category(category_id, title, icon_name)` — `navigation.py`

2.9.4.3 4.3 Relevante Keys (Auszug aus `AppSettings`)

- Allgemein: `theme`, `theme_id`, `model`, `temperature`, `max_tokens`, `think_mode`, \{}ldots{}
- Routing: `auto_routing`, `cloud_escalation`, `cloud_via_local`, `default_role`, `ollama_api_key`, \{}ldots{}

- RAG: `rag_enabled`, `rag_space`, `rag_top_k`, `self_improving_enabled`, `chat_mode`, \{}ldots{}
- Prompts: `prompt_storage_type`, `prompt_directory`, `prompt_confirm_delete`
- Chat: `chat_streaming_enabled`, `chat_context_mode`, `chat_context_detail_level`, `chat_context_include_*`, `chat_context_profile_enabled`, `chat_context_profile`
- Debug: `debug_panel_enabled`, `context_debug_enabled`, `chat_guard_ml_enabled`

Vollst\{}"andige Liste: Methoden `load` und `save` in `app/core/config/settings.py`.

2.9.5 5. Fehler- / Eskalationssicht

Problem	Ursache
Einstellung greift nicht	H\{}"oherpriore Runtime-Policy (Kontext) oder falscher
Nur Context Mode sichtbar, Rest nicht	`chat_context_detail_level` / Includes / Profil haben
Falsche Kategorie nach Deep-Link	Alte `workspace_id`; Mapping in `settings_screen.py` p

2.9.6 6. Wissenssicht

Begriff	Ort
`AppSettings`	`app/core/config/settings.py`
`SettingsBackend`, `InMemoryBackend`	`app/core/config/settings_backend.py`
`SettingsWorkspace`, `_category_factories`	`app/gui/domains/settings/settings_workspace`
`SettingsNavigation`, `DEFAULT_CATEGORIES`	`app/gui/domains/settings/navigation.py`
`SettingsScreen`	`app/gui/domains/settings/settings_screen.py`

2.9.7 7. Perspektivwechsel

Perspektive	Fokus
User	Acht Kategorien; Advanced f\{}"ur Kontextmodus und Debug-Checkboxen.
Admin	Persistenzpfade, Rechte auf Prompt-Verzeichnis, ChromaDB-Ort (RAG).
Dev	Neue Keys immer in `load`/`save`; Panel-Bindings; Tests mit `InMemoryBackend`.

Chapter 3

Workflows

3.1 agent_usage

Endnutzer-Workflow: **Agenten** w\{}"ahlen, **Agent Tasks** nutzen und **Delegation** \{}"uber `/delegate`. Codebasis: `app/agents/`, Workspaces `operations_agent_tasks`, `cc_agents`, Slash-Parser `app/core/commands/chat_commands.py`.

3.1.1 Ziel

Sie nutzen ein **Agentenprofil** oder lassen eine **Delegations**-Bearbeitung mit mehreren Schritten \{}"uber die Agenten-Infrastruktur laufen.

3.1.2 Voraussetzungen

1. Anwendung gestartet, **Ollama** erreichbar (wie beim Chat).
2. Agenten sind in Ihrer Umgebung **angelegt** (Standard: Seed-Profile beim ersten Start — siehe `app/agents/seed_agents.py`; bei leerem Dropdown Admin informieren).
3. Sie kennen den Unterschied: **Agent im Chat** (andere Systemrolle) vs. **'/delegate'** (Orchestrierungspfad).

3.1.3 Schritte A: Agent im Chat verwenden

1. **Operations $\{}rightarrow$ Chat** \{}"offnen (siehe **chat_usage**).
2. In der **Kopfzeile** des Chats (oder dem daf\{}"ur vorgesehenen Steuerelement): **Agent** aus der Liste w\{}"ahlen — falls die Oberfl\{}"ache das anbietet.
3. Nachricht normal eingeben und senden.
4. Das Modell arbeitet mit dem **Systemprompt** und der **Modellzuweisung** des gew\{}"ahlten Profils (`AgentProfile`).

3.1.4 Schritte B: Agent Tasks \{}"offnen

1. Sidebar: **Operations**.
2. In der zweiten Leiste **Agent Tasks** w\{}"ahlen (Workspace **‘operations_agent_tasks‘**).
3. Dort die projektbezogenen Aufgaben und Status einsehen und Aktionen ausf\{}"uhren, die Ihre Version der Oberfl\{}"ache anbietet (Listen, Buttons — ohne hier Einzelfunktionen zu erfinden).

3.1.5 Schritte C: Delegation mit `/delegate`

1. **Operations $\{}rightarrow$ Chat** \{}"offnen.
2. In das Eingabefeld schreiben — **eine Zeile**, beginnend mit:

- **‘/delegate ‘** (Slash, Wort delegate, **Leerzeichen**)
- direkt danach **Ihre Aufgabe in eigenen Worten**, z. B.:

```
/delegate Fasse die letzten drei \{}"Anderungen im Projekt in
drei Bulletpoints zusammen
```

1. Senden.
2. **Ohne Text nach ‘/delegate‘** zeigt die App nur einen **Hinweis zur korrekten Verwendung** — es wird keine Delegationsaufgabe mit Inhalt gestartet (so implementiert in `parse_slash_command`).

3.1.6 Varianten

Situation	Vorgehen
Schnelle Frage ohne Spezialagent	Keinen Agent w\{}"ahlen; Standard-Chat.
Code-fokussiert	Agent ?Code o. \{}"A. w\{}"ahlen, falls vorhanden, oder `/coc`
Gro\{}ss{}e, mehrteilige Aufgabe	`/delegate` mit klarer Aufgabenbeschreibung in **einem** Senden
Agenten verwalten	**Control Center $\{}rightarrow$ Agents** (`cc_agents`) —

3.1.7 Fehlerf\{}"alle

Was Sie sehen	Was Sie tun
Agent-Dropdown leer	App neu starten; **Control Center $\{}rightarr**
`/delegate` ohne Reaktion auf die Aufgabe	Pr\{}"ufen, ob **Text nach** `/delegate` stand; nur
Fehlermeldung ?Unbekannter Befehl	Tippfehler: exakt **‘/delegate‘** schreiben.
Delegation l\{}"auft, Ergebnis unbefriedigend	Aufgabe **sch\{}"arfer** formulieren; k\{}"urzeren
Agent Tasks zeigen nichts	**Projekt** aktiv? Agent Tasks sind projektbezogen,

3.1.8 Tipps

- **Eine klare Aufgabe pro '/delegate'**: Wer soll was liefern, in welchem Format?
- Vor **Delegation** kurz pr\{}"ufen, ob **Chat-Kontext** passt (Workflow **context_control**) — sonst fehlt dem Modell der gew\{}"unschte Bezug.
- F\{}"ur wiederkehrende Agenten-Einstellungen **Control Center $\{}rightarrow$ Agents** mit dem Team abstimmen, nicht jede Session neu erfinden.

3.2 chat_usage

Endnutzer-Workflow: erste Unterhaltung, Modellwahl, Streaming, Slash-Befehle. Orientierung an der GUI unter **Operations $\{}rightarrow$ Chat** (`operations_chat`).

3.2.1 Ziel

Sie f\{}"uhren eine Konversation mit dem Sprachmodell: Nachrichten senden, Antwort erhalten, bei Bedarf Rolle oder Routing anpassen.

3.2.2 Voraussetzungen

1. Die Anwendung ist gestartet (`python -m app` oder `python main.py`).
2. **Ollama** l\{}"auft auf dem Rechner oder dem konfigurierten Server (`ollama serve`).
3. Mindestens ein Modell ist geladen (z. B. Terminal: `ollama pull qwen2.5`).
4. Sie wissen, ob Sie mit einem **Projekt** arbeiten: Projekt im **Project Hub** oder \{}"uber die Kontextleiste im Chat gew\{}"ahlt — oder Sie arbeiten ohne spezifisches Projekt.

3.2.3 Schritte

3.2.3.1 A) Chat-Bereich \{}"offnen

1. Schauen Sie auf die **linke Seitenleiste**.
2. Klicken Sie auf den Bereich **Operations** (nicht Control Center, nicht Settings).
3. In der **zweiten Leiste** (neben dem Hauptfenster) w\{}"ahlen Sie **Chat** — nicht ?Projects, nicht ?Knowledge.

4. Es erscheint der Chat-Workspace: links die **Session-Liste**, in der Mitte die **Konversation**, unten die **Eingabe**.

3.2.3.2 B) Chat ausw\{}"ahlen oder anlegen

1. Links in der Session-Liste einen bestehenden Eintrag anklicken **oder** die Aktion zum **neuen Chat** nutzen (je nach Beschriftung: z. B. ?Neuer Chat / Plus — genaue Bezeichnung in Ihrer Version).

2. Oben in der Mitte sehen Sie ggf. **Projekt**, **Chat-Titel** und optional **Topic** in der **Kontextleiste** \{}"uber der Konversation. So wissen Sie, in welchem Kontext Sie schreiben.

3.2.3.3 C) Modell pr\{}"ufen

1. In der **Kopfzeile** des Chats oder neben der Eingabe: **Modell** aus der Liste w\{}"ahlen.

2. Wenn die Liste leer ist: abbrechen, **Control Center $\{}rightarrow$ Models** oder **Einstellungen $\{}rightarrow$ AI / Models** pr\{}"ufen und Ollama-Status unter **Control Center $\{}rightarrow$ Providers** ansehen (siehe Workflow **settings_usage** und Fehlerf\{}"alle unten).

3.2.3.4 D) Nachricht schreiben und senden

1. Cursor in das **Eingabefeld** unten setzen.

2. Text tippen.

3. **Senden** klicken oder die in der App vorgesehene Tastenkombination nutzen.

4. Wenn **Streaming** eingeschaltet ist (`chat_streaming_enabled` in den Einstellungen), erscheint die Antwort **w\{}"ahrend** der Erzeugung. Wenn aus, erscheint sie **am Ende** in einem St\{}"uck.

3.2.3.5 E) Rolle nur f\{}"ur die n\{}"achste Frage setzen (Slash-Befehl)

1. Zeile beginnt mit **‘/‘** — kein Leerzeichen davor.

2. Beispiele (genau so implementiert in `app/core/commands/chat_commands.py`):

- **‘/think Ihre Frage hier‘** — Denk-Rolle, der Text nach dem Befehl wird als normale Nachricht mit dieser Rolle gesendet.

- **‘/code \{}ldots{}‘**, **‘/research \{}ldots{}‘**, **‘/fast‘**, **‘/smart‘**, **‘/chat‘**, **‘/vision‘**, **‘/overkill‘** — ebenfalls Rollen; mit Text dahinter wird mitgesendet, ohne Text nur Modus-Umschaltung mit kurzer Meldung.

1. **‘/auto on‘** oder **‘/auto off‘** — Auto-Routing.
2. **‘/cloud on‘** oder **‘/cloud off‘** — Cloud-Eskalation (nur sinnvoll, wenn Ihre Umgebung Cloud und Schl\{}"ussel erlaubt).

3.2.4 Varianten

Situation	Was Sie tun
Kurze Alltagsfrage	Direkt tippen, Standardmodell lassen.
Code erkl\{}"aren lassen	`/code` vor die Frage setzen oder Code-Rolle w\{}"ahlen, falls a
Tiefere \{}"Uberlegung	`/think` plus Frage in einer Zeile.
Mehrstufige Aufgabe an Agenten	Workflow **agent_usage** (`/delegate`).

3.2.5 Fehlerf\{}"alle

Was Sie sehen	Was Sie tun
Keine Antwort, endlos wartend	Ollama pr\{}"ufen; **Providers** auf ?online;
Modell-Dropdown leer	Modell mit `ollama pull` laden; App neu sta
Slash-Befehl erscheint als normaler Chat-Text	Zeile muss mit `/` beginnen; Befehl exakt schr
Meldung ?Unbekannter Befehl	Tippfehler; erlaubt sind u. a. `/fast`, `/smart`
Zwei Antworten / seltsames Verhalten beim Senden	Vorherigen Lauf abwarten; ggf. App neu sta

3.2.6 Tipps

- Vor l\{}"angeren Arbeiten **Projekt** und **Chat-Titel** in der Kontextleiste kurz pr\{}"ufen — vermeidet Nachrichten im falschen Chat.
- F\{}"ur wiederkehrende Texte den **Prompt Studio** nutzen (Workflow gesondert).
- Wenn Antworten zu ?allgemein wirken: Workflow **context_control** lesen (Kontextmodus).

3.3 context_control

Endnutzer-Workflow: **Chat-Kontext** steuern — wie viel Hintergrund (Projekt, Chat, ggf. Topic) dem Modell als Strukturinformation mitgegeben wird. Technische Details (Policies, Profil-Overrides) sind in `docs_manual/modules/context/README.` hier nur das, was Sie in der App direkt sehen und einstellen k\{}"onnen.

3.3.1 Ziel

Sie entscheiden, ob das Modell **Projekt- und Chat-Bezug** als zus\{}"atzlichen Systemtext bekommt und in welcher **Darstellungsform** — oder ob dieser Teil **abgeschaltet** ist.

3.3.2 Voraussetzungen

1. Anwendung gestartet.
2. Sie k\{}"onnen **Einstellungen** \{}"offnen (Sidebar **Settings**).
3. Optional: Sie nutzen **Operations $\{}rightarrow$ Chat**, um die **Kontextleiste** \{}"uber der Konversation zu sehen (Projekt, Chat, Topic).

3.3.3 Begriffe in drei Stufen (f\{}"ur Endnutzer)

Die App speichert einen **Chat-Kontext-Modus** (`chat_context_mode`). In der Oberfl\{}"ache stellen Sie ihn unter **Einstellungen $\{}rightarrow$ Advanced** im Feld **?Chat-Kontext-Modus“** ein (`AdvancedSettingsPanel` — Auswahlliste mit **neutral**, **semantic**, **off**).

Einstellung	Was es f\{}"ur Sie bedeutet
off	Es wird **kein** strukturierter Hintergrund aus Projekt/Chat/Topic in den Prompt eing
neutral	Strukturierter Hintergrund wird **knapp/sachlich** formuliert mitgegeben.
semantic	Strukturierter Hintergrund wird **ausf\{}"uhrlicher** formuliert mitgegeben.

Hinweis: Weitere Schalter wie **Detailstufe** (minimal / standard / full) und **welche Felder** (Projekt/Chat/Topic) einflie\{}ss{}en, existieren in der Konfiguration (`AppSettings`), sind in den **Einstellungs-Panels** aber **nicht** jedes Mal als eigene Felder sichtbar — nur dieser **Modus** ist dort direkt gebunden. Wenn sich etwas ?nicht wie erwartet verh\{}"alt, kann eine **andere Regel** in der App Vorrang haben (z. B. Profil oder Policy); das kl\{}"art der Support mit den Debug-Ansichten.

3.3.4 Schritte: Kontextmodus \{}"andern

1. Sidebar: **Settings** (Einstellungen) anklicken.
2. Links die Kategorie **Advanced** w\{}"ahlen (nicht Appearance, nicht AI / Models).
3. Im Formular **?Chat-Kontext-Modus:“** die Liste \{}"offnen.
4. **off**, **neutral** oder **semantic** w\{}"ahlen.

5. Die App speichert typischerweise **automatisch** beim Wechsel (an `AppSettings` gebunden).

3.3.5 Wann off?

W\{}"ahlen Sie **off**, wenn:

- Sie **allgemeine** Fragen stellen, die **nicht** vom aktuellen Projekt oder Chat-Titel abh\{}"angen sollen.
- Sie vermeiden wollen, dass **interne Namen** (Projekt, Titel) das Modell in eine Richtung lenken.
- Sie testen wollen, ob Antworten ohne diesen Zusatztext **neutraler** werden.

Konsequenz: Die **Kontextleiste** im Chat kann weiter Projekt/Chat anzeigen — das ist nur **Anzeige**. Der Modus **off** bedeutet: dieser Block wird **nicht** als strukturierter Systemkontext injiziert.

3.3.6 Wann semantic?

W\{}"ahlen Sie **semantic**, wenn:

- Das Modell **Projekt- und Chatbezug** nutzen soll, um Antworten **passender** zu formulieren (z. B. ?in diesem Repo, ?zu diesem Chat).
- Sie die **ausf\{}"uhrlichere** Darstellungsvariante wollen (gegen\{}"uber **neutral**).

Standard in der Konfiguration ist **semantic**, falls nichts anderes gesetzt war (`AppSettings.load`).

3.3.7 Wann neutral?

W\{}"ahlen Sie **neutral**, wenn:

- Sie Hintergrund **mitgeben** wollen, aber die **k\{}"urzere/sachlichere** Formulierung bevorzugen (z. B. weniger erkl\{}"arender Text im Systemblock).

3.3.8 Kontextleiste im Chat (ohne Einstellungen zu \{}"andern)

1. **Operations $\{}rightarrow$ Chat** \{}"offnen.
2. \{}"Uber der Konversation: Anzeige **Projekt**, **Chat**, ggf. **Topic**.

3. **Klicks** auf diese Eintr\{}"age: dienen der **Navigation/Umbenennung** je nach Implementierung — **nicht** dem Umschalten von **off/neutral/semantic** (das bleibt unter **Advanced**).

3.3.9 Varianten

Situation	Empfehlung (fachlich)
Support-Mail formulieren, ohne Projektbezug	**off**
Feature in genau diesem Projekt besprechen	**semantic** (oder **neutral**)
Datenschutz: keine Projektnamen ins Modell	**off**
Vergleich: gleiche Frage mit/ohne Kontext	Erst **off**, dann **semantic** testen

3.3.10 Fehlerf\{}"alle

Symptom	Ursache (aus Nutzersicht)
Modell ?ignoriert das Projekt	Modus **off** aktiv
Modell halluziniert Projektbezug	Trotz **off** kommen noch **RAG** oder **lange Chat-Histo**
\{}"Anderung wirkt nicht	Eine **h\{}"oherpriore** Regel in der App \{}"uberschre
Nur **eine** Stelle zum Umstellen gefunden	**Richtig** — nur **Chat-Kontext-Modus** ist dort an die

3.3.11 Tipps

- Nach dem Umstellen des Modus **eine neue kurze Frage** stellen, um den Effekt zu pr\{}"ufen — alte Antworten \{}"andern sich nicht r\{}"uckwirkend.
- **Kontext-Inspection aktivieren** (Checkbox unter **Advanced**) nur, wenn Ihre Organisation das erlaubt — f\{}"ur Diagnose, nicht f\{}"ur den Alltag n\{}"otig.

3.4 settings_usage

Endnutzer-Workflow: **Einstellungen** finden, die **acht Kategorien** durchlaufen und die wichtigsten Schalter setzen. Codebasis: `app/gui/domains/settings/navigation.py`, `settings_workspace.py`, `app/core/config/settings.py`.

3.4.1 Ziel

Sie passen Darstellung, Modell, Daten, Datenschutz, erweiterte Optionen sowie projekt- und workspacebezogene Seiten an — ohne die technischen Schl\{}"usselnamen kennen zu m\{}"ussen.

3.4.2 Voraussetzungen

1. Anwendung gestartet.
2. Sie haben die Berechtigung, Einstellungen auf diesem Rechner zu \{}"andern (kein schreibgesch\{}"utztes Nutzerprofil auf OS-Ebene, das QSettings blockiert).

3.4.3 Schritte: Einstellungen \{}"offnen

1. In der **linken Sidebar** ganz nach unten bzw. zum Eintrag **Settings** (Einstellungen) gehen.
2. Klicken Sie **Settings** — der **volle Bildschirm** wechselt in den Einstellungsmodus (nicht nur ein kleines Pop-up).
3. Layout: **links** die **Kategorienliste**, **Mitte** der Inhalt, **rechts** optional ein kurzer Hilfetext (`SettingsHelpPanel`).

3.4.4 Schritte: Kategorien der Reihe nach (8 St\{}"uck)

Die **Reihenfolge** entspricht der Navigation in der App:

Nr.	Kategorie (Anzeigename)	Typische Inhalte (was Sie dort erwarten)
1	**Application**	Globale Anwendungsoptionen.
2	**Appearance**	**Theme**, Darstellung — hier wechseln Sie z. B. hell/dunkel bzw.
3	**AI / Models**	**Standardmodell**, Temperatur, Token-Grenzen, Denkmodus — a
4	**Data**	Datenbezogene Optionen (Speicherorte, Pfade — je nach Panel-In
5	**Privacy**	Datenschutzbezogene Schalter.
6	**Advanced**	**Debug-Panel**, **Kontext-Inspection**, **Chat-Kontext-Modus**
7	**Project**	Einstellungen mit **Projektbezug**.
8	**Workspace**	Einstellungen mit **Arbeitsbereichsbezug**.

IDs im Code (falls Support danach fragt): `settings_application`, `settings_appearance`, `settings_ai_models`, `settings_data`, `settings_privacy`, `settings_advanced`, `settings_project`, `settings_workspace`.

3.4.5 Schritte: \{}"Anderung speichern

1. In den meisten Panels werden \{}"Anderungen beim Umstellen **sofort** an `AppSettings` \{}"ubergeben und mit **'save()'** persistiert (z. B. Checkboxen und Combos in `AdvancedSettingsPanel`).
2. Schlie\{}ss{}en Sie die Einstellungen \{}"uber die **Navigation zur\{}"uck** zum Arbeitsbereich (Sidebar: wieder **Operations** oder **Chat** w\{}"ahlen).

3.4.6 Varianten

Bedarf	Wo hin
Nur Theme \{}"andern	**Appearance** — fertig.
Ollama langsam / falsches Modell	**AI / Models** + **Control Center $\{}rightarrow$ Model**
RAG an/aus, Space, Top-K	In den Panels, die **RAG**-Schalter anzeigen — oft in Zusamme
Chat ohne Projekt-Kontext	**Advanced** $\{}rightarrow$ Chat-Kontext-Modus **off** (Workfl
Prompts nur im Dateiordner	Schl\{}"ussel `prompt_storage_type` / `prompt_directory` —

3.4.7 Fehlerf\{}"alle

Symptom	Was Sie tun
Einstellung springt nach Neustart zur\{}"uck	Support: pr\{}"ufen, ob `save()` f\{}"ur dieses Fel
Kategorie leer oder nur Platzhalter	**Project / Workspace** k\{}"onnen in fr\{}"uhen
RAG-Schalter nicht auffindbar	Unter **Data** oder **AI / Models** suchen; ggf. Doku
Nach Theme-Wechsel wirkt etwas ?kaputt	App einmal neu starten; Theme-Registry in `Appea`
Sie kommen nicht zur\{}"uck zum Chat	Sidebar: **Operations** $\{}rightarrow$ **Chat** — Se

3.4.8 Tipps

- **Vor Modellwechsel** unter **AI / Models** kurz **Control Center $\{}rightarrow$ Providers** pr\{}"ufen — sonst wirkt ein neues Modell ?tot, obwohl die Einstellung gespeichert ist.
- **Advanced** nicht f\{}"ur alle Nutzer freigeben: Debug und Kontext-Inspection sind f\{}"ur **Entwicklung und QA** gedacht.
- F\{}"ur **RAG** und **Prompts** die speziellen Hilfeseiten im Ordner `help/settings/` nutzen (`settings_rag.md`, `settings_prompts.md`).

Chapter 4

Rollen

4.1 Admin

Handbuch f\{}"ur Betrieb, Konfiguration und Kontrolle der Linux-Desktop-Chat-Instanz. Schritte beziehen sich auf die implementierte GUI (`app/gui/`), Einstellungen (`AppSettings` in `app/core/config/settings.py`) und Provider unter `app/providers/`.

4.1.1 Aufgaben

- **Ollama** bereitstellen: Dienst laufen lassen, Modelle vorab laden (`ollama pull \{}ldots{}`), Erreichbarkeit pr\{}"ufen.
- **Einstellungen** der App \{}"uberwachen: Organisation `OllamaChat` / `LinuxDesktopChat` in **QSettings** (Endnutzerdoku: `help/settings/settings_overview.md`).
- **Provider- und Modellzugriff** pr\{}"ufen: **Control Center $\{}rightarrow$ Providers** und **Models** entsprechen `cc_providers` / `cc_models`.
- **RAG-Betrieb:** `chromadb` installiert (`requirements.txt`), Datenverzeichnis typisch `./chroma_db`; Embedding-Modell in Ollama (z. B. `nomic-embed-text`) bereitstellen; Indexierung z. B. \{}"uber `scripts/index_rag.py` (siehe `help/troubleshooting/troubleshooting.md`).
- **Cloud:** `ollama_api_key`, `cloud_escalation`, `cloud_via_local` in den App-Einstellungen abstimmen.
- **Backups:** SQLite-Datei f\{}"ur Chat-Historie (`chat_history.db`), ChromaDB-Ordner, ggf. Prompt-Verzeichnis bei `prompt_storage_type=directory`.

4.1.2 Typische Workflows

4.1.2.1 Erreichbarkeit Ollama pr\{}"ufen

1. Auf dem Zielrechner: `ollama serve` (oder Dienst-Unit).

2. Pr\{}"ufen: `curl http://localhost:11434/api/tags` (oder Ihr Endpoint).

3. In der App: **Control Center $\{}rightarrow$ Providers** — Status **online** erwarten.

4. Wenn offline: Firewall, Port **11434**, falsche URL in der Provider-Konfiguration pr\{}"ufen.

4.1.2.2 Standardmodell und Modellliste

1. **Control Center $\{}rightarrow$ Models:** Liste aktualisieren; Standardmodell setzen (speichert \{}"uber Backend in `AppSettings.model`, siehe Modul **providers** / **settings**).

2. Fehlende Modelle: auf dem Ollama-Host `ollama pull <name>`.

4.1.2.3 Einstellungskategorien durchgehen (8 St\{}"uck)

Unter **Settings** die linke Navigation nutzen — IDs und Titel aus `app/gui/domains/settings/naviga`

Kategorie (UI)	ID
Application	`settings_application`
Appearance	`settings_appearance`
AI / Models	`settings_ai_models`
Data	`settings_data`
Privacy	`settings_privacy`
Advanced	`settings_advanced`
Project	`settings_project`
Workspace	`settings_workspace`

Im **Advanced**-Bereich: **Chat-Kontext-Modus** (`chat_context_mode`), **Kontext-Inspection** (`context_debug_enabled`), **Agent Debug Tab** (`debug_panel_enabled`) — nur f\{}"ur vertrauensw\{}"urdige Nutzer aktivieren.

4.1.2.4 RAG-Space und Top-K

1. **Settings:** `rag_enabled`, `rag_space`, `rag_top_k` pr\{}"ufen (`AppSettings`).

2. Index f\{}"ur den gew\{}"ahlten Space erzeugen/aktualisieren (Skript-Pfad im Troubleshooting-Artikel).

3. Bei leeren Treffern: falschen Space, leeren Index oder fehlendes Embedding-Modell ausschlie\{}ss{}en.

4.1.2.5 Kontrolle bei Supportf\{}"allen

1. **Runtime / Debug** in der App: Logs, Metriken, **LLM Calls**, **Agent Activity** (Workspace-IDs z. B. `rd_logs`, `rd_llm_calls`, `rd_agent_activity` in `runtime_debug_screen.py`).
2. **QA & Governance** bei internem Qualit\{}"atsprozess: **Incidents**, **Replay Lab** (`qa_incidents`, `qa_replay_lab`).

4.1.3 Genutzte Module

Modul	Admin-Fokus
settings (`../../modules/settings/README.md`)	Acht Kategorien, alle persistierten Keys.
providers (`../../modules/providers/README.md`)	Lokaler/Cloud-Ollama, Status-Anzeige.
rag (`../../modules/rag/README.md`)	ChromaDB, Spaces, Index-Pipeline.
gui (`../../modules/gui/README.md`)	Welche Screens/Workspaces f\{}"ur Kontroll
context (`../../modules/context/README.md`)	Kontextmodus, Overrides — bei ?falschem V
chains (`../../modules/chains/README.md`)	Priorit\{}"atskette bei Kontext-Konflikten.

4.1.4 Risiken

- **Cloud-Keys** in Klartext in QSettings: Zugriff auf Nutzerkonten / Arbeitsplatz absichern.
- **ChromaDB-Verzeichnis besch\{}"adigt:** Neuindexierung n\{}"otig; Datenverlust des Vektorstores.
- **Kontext-Inspection / Debug** f\{}"ur alle aktiviert: erh\{}"ohte Sichtbarkeit interner Abl\{}"aufe — nur f\{}"ur berechtigte Personen.
- **Profile/Policy-Overrides:** Endnutzer-\{}"Anderungen an `chat_context_mode` k\{}"onnen durch h\{}"oherpriore Quellen \{}"uberstimmt werden — Support muss `ChatService._resolve_context_configuration` logisch kennen (siehe Modul **context**).

4.1.5 Best Practices

- Modelle und Ollama-Version dokumentieren; nach Updates Rauchtest: Chat senden, Provider gr\{}"un.
- RAG: feste Spaces pro Umgebung (Entwicklung/Test/Produktion) und klare Index-Jobs.
- Backups vor Migrationen (DB + `chroma_db` + relevante Verzeichnisse).
- F\{}"ur produktive Nutzer **Cloud** und **RAG**-Policies schriftlich festlegen.

4.2 Business

Kurzfassung f\{}"ur Entscheidungstr\{}"ager: wof\{}"ur die Anwendung eingesetzt wird, welche Bausteine das erm\{}"oglichen und welche betrieblichen und fachlichen Risiken aus der Produktarchitektur folgen. Inhalte spiegeln die implementierten Module wider, keine zus\{}"atzlichen Produktversprechen.

4.2.1 Aufgaben

- Einsch\{}"atzen, ob die **lokale Desktop-Plattform** (PySide6 + Ollama + optional RAG/Agenten) zu IT- und Arbeitsmodellen passt.
- **Nutzen** gegen\{}"uber reinem Web-Chat abw\{}"agen: Datenverarbeitung auf dem Arbeitsplatz/Server, konfigurierbare Modell- und Wissensanbindung.
- **Risiken** mit IT (Provider, Cloud, Speicherorte) und Fachbereichen (Prompt-Qualit\{}"at, RAG-Genauigkeit) kl\{}"aren.

4.2.2 Typische Workflows (organisatorisch)

4.2.2.1 Einf\{}"uhrung f\{}"ur Wissensarbeit mit internen Dokumenten

1. IT stellt **Ollama** und Basis-Modelle bereit.
2. Fachbereich definiert, welche **Dokumente** in den RAG-Index d\{}"urfen (`Knowledge`-Workspace, `rag_space`).
3. Nutzer arbeiten in **Operations $\{}rightarrow$ Chat** mit aktiviertem RAG, wenn die Policy das erlaubt.
4. Qualit\{}"atssicherung: Stichproben auf Halluzinationen und veraltete Chunk-Inhalte.

4.2.2.2 Einf\{}"uhrung f\{}"ur spezialisierte Bearbeitung (Agenten / Delegation)

1. **Control Center $\{}rightarrow$ Agents:** Profile pflegen (Rollen, Modelle).
2. Nutzer nutzen **Agent Tasks** oder `/delegate Aufgabe` im Chat f\{}"ur Aufgaben, die die Agenten-Pipeline abdeckt (`app/agents/`).
3. Kl\{}"arung, welche Aufgaben **nicht** automatisiert werden d\{}"urfen (Compliance, Freigaben).

4.2.2.3 Standardisierung von Texten (Prompts)

1. **Prompt Studio** als zentrale Vorlagenbibliothek nutzen (`operations_prompt_studio`).
2. Speicherort festlegen: Datenbank vs. Verzeichnis (`prompt_storage_type`, `prompt_directory`).
3. Schulung: Vorlagen versionieren und verbindlich machen, wo sinnvoll.

4.2.3 Genutzte Module (Nutzen in Gesch\{}"aftssprache)

Modul	Nutzen / Einsatzm\{}"oglichkeit
chat (`../../modules/chat/README.md`)	Zentraler Dialog mit Sprachmodellen; Stream
context (`../../modules/context/README.md`)	Steuerbarer Projekt-/Chat-Bezug im Prompt
rag (`../../modules/rag/README.md`)	Antworten mit Bezug zu **euren** Dokumenten
agents (`../../modules/agents/README.md`)	Spezialprofile und Mehrschritt-Orchestrierung
prompts (`../../modules/prompts/README.md`)	Einheitliche Kommunikations- und Prozessvo
providers (`../../modules/providers/README.md`)	Lokaler Betrieb; optional Cloud-Ollama mit
settings (`../../modules/settings/README.md`)	Zentrale Schalter f\{}"ur Modell, RAG, Clou
gui (`../../modules/gui/README.md`)	Eine Oberfl\{}"ache f\{}"ur Chat, Wissen, S

4.2.4 Risiken

Risiko	Herkunft (Produkt)
Falsche oder veraltete RAG-Antworten	Retrieval und Chunking liefern nur Ausschnitte; ke
Cloud-\{}"Ubermittlung	Bei aktivierter Cloud-Eskalation und g\{}"ultigem
Kein impliziter Projektbezug	Kontextmodus **Aus** oder Overrides — Modell ?sieh
Delegations- und Agentenfehler	Automatisierte Pfade k\{}"onnen fehlschlagen oder
Abh\{}"angigkeit von Ollama	Ausfall oder Performance des Dienstes blockiert Ch
Technische Komplexit\{}"at	QA-/Runtime-Workspaces zeigen Systemtiefe — f\

4.2.5 Best Practices

- **Policy schriftlich:** wann RAG, wann Cloud, wann Kontext **Aus** f\{}"ur sensible Themen.
- **Pilot:** begrenzte Nutzergruppe, definierte Use Cases (Chat-only, Chat+RAG, Agenten).
- **Governance:** Nutzung der **QA & Governance**-Workspaces (`qa_*`) in gr\{}"o\{}ss{}eren Organisationen mit IT-Qualit\{}"atsprozess abstimmen.
- **Schulung:** Slash-Commands und Kontextleiste (Projekt/Chat) erkl\{}"aren — reduziert Supportaufwand.

4.3 Entwickler

Handbuch f\{}"ur Arbeit am Repository: Architektur-Anker, Erweiterungspunkte und Debugging. Pfade beziehen sich auf `app/` wie in `docs_manual/modules/` und `docs/ARCHITECTURE.md`.

4.3.1 Aufgaben

- **Architektur** verstehen: GUI (`app/gui/`) $\{}rightarrow$ Services (`app/services/`) $\{}rightarrow$ Kontext (`app/chat/`, `app/context/`) $\{}rightarrow$ Settings (`app/core/config/`) $\{}rightarrow$ Provider (`app/providers/`), LLM (`app/llm/`), Agenten (`app/agents/`), RAG (`app/rag/`).
- **Features erweitern:** neue Settings-Kategorien, Slash-Commands, Workspaces, Services anbinden.
- **Fehler finden:** Chat-Pfad, Provider, RAG, Async/Signals, Replay/Repro f\{}"ur Kontext.
- **Tests ausf\{}"uhren:** `tests/README.md` — pytest mit Markern `unit`, `integration`, `live`, `ui`.

4.3.2 Typische Workflows

4.3.2.1 Repository lokal starten

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python -m app
```

Alternativen: `python main.py`, `python run_gui_shell.py` (`app/__main__.py` delegiert).

4.3.2.2 Neuen Screen oder Workspace anbinden

1. Screen: Factory in `app/gui/bootstrap.py` bei `get_screen_registry().register(NavArea.\\{}ldots{})` eintragen (`NavArea` in `app/core/navigation/nav_areas.py`).
2. Workspace innerhalb eines bestehenden Screens: analog zu `operations_screen.py` — Tuple `(workspace_id, WorkspaceClass)` zum `QStackedWidget` hinzuf\{}"ugen; Navigation in `*Nav` erweitern.
3. Breadcrumbs: `get_breadcrumb_manager().set_workspace` wird in den Screens bereits bei Workspace-Wechsel aufgerufen — gleiches Muster \{}"ubernehmen.

4.3.2.3 Settings-Kategorie erg\{}"anzen

1. Widget-Klasse unter `app/gui/domains/settings/categories/`.
2. In `settings_workspace.py`: `_category_factories["settings_<id>"] = YourCategory`.
3. In `navigation.py`: `register_settings_category` oder `DEFAULT_CATEGORIES` um Eintrag erweitern.
4. Keys in `AppSettings.load` / `save` (`app/core/config/settings.py`), wenn neue Persistenz n\{}"otig.

4.3.2.4 Slash-Command hinzuf\{}"ugen

1. `app/core/commands/chat_commands.py`: in `parse_slash_command` verzweigen; ggf. `ROLE_COMMANDS` oder neue Verbzweigung.
2. `SLASH_COMMANDS` Liste aktualisieren.
3. Tests f\{}"ur Parser und Chat-Integration erg\{}"anzen.

4.3.2.5 Kontext-Debugging

1. In der App: **Settings $\{}rightarrow$ Advanced** — `context_debug_enabled` aktivieren.
2. Code: `ChatService._resolve_context_configuration` (`app/services/chat_service.py`), Traces/`ChatContextResolutionTrace` (`app/chat/context_profiles.py`).
3. Headless: `python -m app.cli.context_replay <json>`, `python -m app.cli.context_repro_run <json>` (`app/cli/`).
4. Explainability: `app/context/explainability/`, `app/services/context_explain_service.p` `policy_chain` in `context_explanation_serializer.py`.

4.3.2.6 Provider-/Modellprobleme eingrenzen

1. `app/providers/ollama_client.py` und `local_ollama_provider.py` / `cloud_ollama_provider.py`.
2. `app/services/chat_service.py` Sendepfad und Fehlerbehandlung.
3. Integrationstests unter `tests/integration/`, Chaos unter `tests/chaos/`.

4.3.2.7 GUI-Pfade nicht mit Alt-Repo verwechseln

GUI liegt unter **'app/gui/'**. Verzeichnis **'app/ui/'** existiert im aktuellen Stand nicht (`docs/DOC_GAP_ANALYSIS.md`).

4.3.3 Genutzte Module

Modul	Dev-Fokus
gui (`../../modules/gui/README.md`)	`bootstrap.py`, `workspace_host`, Screen- und
chat (`../../modules/chat/README.md`)	`ChatService`, Guard, Streaming-Flag.
context (`../../modules/context/README.md`)	Aufl\{}"osung, Limits, Replay, Serializer.
settings (`../../modules/settings/README.md`)	`AppSettings`, Backend, Panel-Bindings.
providers (`../../modules/providers/README.md`)	Ollama-Clients.
agents (`../../modules/agents/README.md`)	Planner, Delegation, Execution.
rag (`../../modules/rag/README.md`)	Retriever, vector_store, pipeline.
prompts (`../../modules/prompts/README.md`)	Repository, StorageBackend.
chains (`../../modules/chains/README.md`)	Priorit\{}"aten, Pipelines, Delegation.

Zus\{}"atzlich: **‘docs/DEVELOPER_GUIDE.md‘**, **‘docs/ARCHITECTURE.md‘**, **‘tools/generate_system_map.py‘** f\{}"ur aktuelle `docs/SYSTEM_MAP.md`.

4.3.4 Risiken

- **Async/Qt:** Blocking-Calls in UI-Threads vermeiden; Muster in bestehenden Workspaces \{}"ubernehmen.
- **Kontext-\{}"Anderungen ohne Tests:** Regression in Golden/QA-Tests (`tests/`, `docs/qa/`).
- **Settings-Drift:** Neuer Key ohne `save()` $\{}rightarrow$ Wert geht bei Neustart verloren.
- **Inspector-Stale-Content:** `content_token` / `prepare_for_setup` bei Inspector-Updates beachten (`OperationsScreen`, `RuntimeDebugScreen`).

4.3.5 Best Practices

- Vor gr\{}"o\{}ss{}eren GUI-\{}"Anderungen `pytest tests/ui` und relevante `integration`-Tests laufen lassen.
- Kontext\{}"anderungen mit Replay-JSON und Repro-F\{}"allen absichern (`app/context/replay/`).
- \{}"Offentliche Erweiterungspunkte nutzen: `register_settings_category_widget`, `register_all_screens`-Muster, CLI-Module als Referenz f\{}"ur deterministische Ausgaben.
- Nach Struktur\{}"anderungen `python3 tools/generate_system_map.py` ausf\{}"uhren und Diff pr\{}"ufen.

4.4 Fachanwender

Handbuch f\{}"ur die t\{}"agliche Nutzung der Desktop-App ohne technische Implementierungsdetails. Begriffe und Schritte sind an der realen Oberfl\{}"ache und den Modulen unter `docs_manual/modules/` ausgerichtet.

4.4.1 Aufgaben

- Im Bereich **Operations** Chats f\{}"uhren, Inhalte mit dem Modell erarbeiten.
- Unter **Knowledge** Dokumente verwalten und f\{}"ur RAG-basierte Antworten nutzen (wenn aktiviert).
- Im **Prompt Studio** Vorlagen pflegen und im Arbeitsfluss verwenden.
- **Agent Tasks** und **Agentenwahl im Chat** nutzen, wenn Ihre Umgebung das vorsieht.
- **Einstellungen** \{}"offnen: Erscheinungsbild, KI/Modelle, ggf. erweiterte Optionen inkl. **Chat-Kontext-Modus**.
- \{}"Uber **Control Center** pr\{}"ufen, ob Modelle und Provider erreichbar sind, bevor Sie chatten.

4.4.2 Typische Workflows

4.4.2.1 Neuen Chat starten und schreiben

1. Sidebar: **Operations** w\{}"ahlen.
2. In der linken Leiste **Chat** aktivieren (Workspace **Chat**).
3. Session w\{}"ahlen oder neue Session anlegen.
4. Unten Text eingeben und senden.
5. Modell oder Agent \{}"uber die Kopfzeile des Chats w\{}"ahlen, soweit angeboten.

4.4.2.2 Rolle oder Routing per Eingabezeile

Am Anfang der Nachricht, direkt nach den Regeln der App:

- `/think \{}ldots{}`, `/code \{}ldots{}`, `/research \{}ldots{}` usw. setzen die **Rolle** f\{}"ur diese Nachricht (Rest der Zeile wird mitgesendet).
- `/auto on` oder `/auto off` schalten **Auto-Routing**.

- `/cloud on` oder `/cloud off` steuern **Cloud-Eskalation** (nur wirksam, wenn Ihre Organisation Ollama-Cloud und Schl\{}"ussel freigibt).
- `/delegate Aufgabe` startet die **Delegations**-Bearbeitung mit dem Text nach dem Befehl — ohne Text erscheint nur ein Hinweis.

(Implementierung: `app/core/commands/chat_commands.py`.)

4.4.2.3 RAG nutzen

1. **Operations $\{}rightarrow$ Knowledge:** Quellen anlegen oder aktualisieren, Indexierung ausf\{}"uhren (wie in Ihrer Umgebung vorgesehen, z. B. \{}"uber Skripte aus dem Projekt).
2. In den **Einstellungen** RAG aktivieren und **Space** sowie **Top-K** pr\{}"ufen (`rag_enabled`, `rag_space`, `rag_top_k`).
3. Im **Chat** Frage stellen, die zu den indexierten Inhalten passt.

4.4.2.4 Kontext verstehen (ohne Technik)

Der **Chat-Kontext** ist zus\{}"atzlicher Hintergrund, den die App dem Modell mitgeben kann: z. B. zu welchem **Projekt** und welchem **Chat** die Nachricht geh\{}"ort. Dar\{}"uber gibt die Leiste **\{}"uber** der Konversation Auskunft (Projektname, Chat-Titel, ggf. Topic).

Drei Stufen ?wie viel Hintergrund“ (vereinfacht):

- **Aus:** Es wird kein solcher Struktur-Hintergrund mitgeschickt. Das Modell sieht nur das, was Sie in den Nachrichten schreiben (plus ggf. RAG, Prompts, Agenten-Systemprompt).
- **Neutral:** Hintergrund wird sachlich/knapp formuliert mitgegeben.
- **Semantisch:** Hintergrund wird ausf\{}"uhrlicher mitgegeben.

Wo Sie das umstellen: **Einstellungen $\{}rightarrow$ Advanced $\{}rightarrow$ Chat-Kontext-Modus** (`AdvancedSettingsPanel`).

Wichtig: Wenn Antworten ?ohne Bezug zum Projekt wirken, kann der Modus **Aus** aktiv sein — oder eine andere Regel in der App hat Vorrang vor Ihrer manuellen Einstellung (das kl\{}"aren ggf. Admins oder Support \{}"uber die QA-/Debug-Ansichten).

4.4.2.5 Typische Fehler und was Sie tun k\{}"onnen

Symptom	Was pr\{}"ufen
Keine Antwort / dauert sehr lange	**Control Center $\{}rightarrow$ Providers:**
Modellliste leer	Modell mit `ollama pull \{}ldots{}` laden; App
RAG liefert nichts	Unter **Knowledge** indexiert? RAG in den Einstell
Slash-Befehl wird als normaler Text gesendet	Zeile muss mit `/` beginnen; Tippfehler im Befehlsna
`/delegate` passiert nichts Sichtbares	Kein Text nach `/delegate` eingegeben — es ersche
Chat wirkt ?ohne Projektbezug	Kontextmodus **Aus**; oder Projekt/Chat in der Kon

4.4.3 Genutzte Module

Modul	Nutzen f\{}"ur Sie
gui (`../../modules/gui/README.md`)	Navigation, Operations, Chat-, Knowledge-,
chat (`../../modules/chat/README.md`)	Senden, Streaming, Slash-Commands.
context (`../../modules/context/README.md`)	Bedeutung von Kontextmodus (fachlich verei
prompts (`../../modules/prompts/README.md`)	Prompt Studio.
rag (`../../modules/rag/README.md`)	Knowledge-Workspace und RAG-Schalter.
agents (`../../modules/agents/README.md`)	Agent Tasks, Agentenwahl, `/delegate`.
settings (`../../modules/settings/README.md`)	Alle Einstellungskategorien inkl. Advanced.
providers (`../../modules/providers/README.md`)	Providers-Workspace (Status).

4.4.4 Risiken

- **Falsche Annahmen zum Kontext:** Bei Modus **Aus** oder eingeschr\{}"ankten Includes ?wei\{}ss{} das Modell nicht automatisch alles \{}"uber Projekt/Chat.
- **Cloud und Daten:** Wenn **Cloud** eingeschaltet ist und Ihre Organisation Cloud-Modelle nutzt, verlassen Anfragen das lokale Ollama-Szenario — Kl\{}"arung mit Admin/Richtlinien.
- **RAG-Qualit\{}"at:** Antworten h\{}"angen von Index, gew\{}"ahltem Space und Dokumentqualit\{}"at ab.

4.4.5 Best Practices

- Vor l\{}"angeren Sessions **Provider-Status** und **Standardmodell** pr\{}"ufen.
- F\{}"ur wiederkehrende Texte **Prompt Studio** nutzen statt jedes Mal neu zu formulieren.
- Bei Themen mit internen Dokumenten **Knowledge** pflegen und RAG gezielt aktivieren.

- Wenn Antworten zu allgemein sind: Kontextmodus nicht auf **Aus**, sofern Ihre Richtlinien Projektbezug erlauben.